

## Pointer arithmetic, initialization, and pointer parameters

1. What will the output of this code be?

```
const int SIZE = 4;
int results[SIZE] = { 54, 44, 34, 22 };
int *resultsPtr = nullptr;
resultsPtr = results + 2;
cout << *resultsPtr << endl;
```

34

2. Assume ptr is a pointer to an int and holds the address 12000. On a system with 4-byte integers, what address will be in ptr after the following statement?

```
ptr += 10;
```

Answer:

After the statement, the pointer will point to a place  $10 * 4$  bytes = 40 bytes forward in contiguous memory, and the new address will be  $12000 + 40 = 12040$ .

3. Could you show this with code?

```
int x;
int *ptr = &x;
printf("Old address: %d\n", ptr); // this will generate a warning
ptr += 10;
printf("New address: %d\n", ptr); // this will generate a warning
```

```
Old address: -1025782548
New address: -1025782508
```

4. Assume pint is a pointer-to-int variable. Is each of the following statements valid or invalid? If any is invalid, why?

```
A) pint++;
B) --pint;
C) pint /= 2;
D) pint *= 4;
E) pint += x; // x is an int
```

Solution:

```
int x;
int *pint;
pint++;
--pint;
pint += x;
```

```
// pint /= 2; // not valid
// pint *= x; // not valid
```

5. In the following code, is the pointer variable definition legal or illegal?

```
int seats[5] = {2,4,5,7,6};
int *seats = seats;
```

Answer:

It's illegal since an array name (seats) stands for a constant pointer by the same name so this counts as a double declaration, and if you took the int away, \*seats = seats is a conflict because the compiler cannot convert int\* into int. The way to fix it is to give the pointer variable a different name: seatsPtr.

6. In the following code, is the assignment statement in the 2nd line legal or illegal?

```
double dollars;
int dollarsPtr = &dollars;
```

Answer:

It's illegal since the right hand value is an address (of the double variable dollars), which expects to be stored in a pointer-to-double of the data type double\*.

Solution:

```
double dollars;
double *dollarsPtr = &dollars;
```

7. Is each of the following pointer definitions and initializations valid or invalid? If any is invalid, why?

```
A) int ivar;
   int *iptr = &ivar;

B) int ivar, *iptr = &ivar;

C) float fvar;
   int *iptr = &fvar;

D) int nums[50], *iptr = nums;

E) int *iptr = &ivar;
   int ivar;
```

Solution:

```
// A)
int ivar;           // integer variable
```

```

int *iptr = &ivar;    // address stored in pointer-to-int
// B)
int ivar2, *iptr2 = &ivar; // OK - pointer-to-int stores address
// C)
float fvar;           // floating-point variable
//int *iptr3 = &fvar; // ILLEGAL: pointer-to-int cannot store
// floating-point variable address
// D)
int nums[50], *iptr4 = nums; // pointer-to-int stores address of
                             // nums[0], nums is constant pointer
// E)
int *iptr5 = &ivar;    // ILLEGAL - ivar5 not defined yet - will
// compile!
int ivar5;

```

8. Will this function compile?

```

void getNumber(int *input) {
    cout << "Enter an integer number: ";
    cin >> input;
}
int main(){int num; getNumber(num);}

```

Solution:

```

void getNumber(int *input) {
    cout << "Enter an integer number: ";
    cin >> *input; // `input` is a pointer that holds an address
}
int main(){int num; getNumber(&num);} // address-of num needs to
be passed

Enter an integer number:

```

9. Rewrite this pointer-based function to use a reference parameter instead. Output is 10.

```

void increment(int *ptr) { (*ptr)++; }

int main() {
    int x = 9;
    increment(&x);
    cout << x;
}

```

Solution:

```

void increment(int &x) { x++; }

int main() {

```

```

    int x = 9;
    increment(x);
    cout << x;
}

```

10. The function `copyInts` copies an array `src` into an array `dst`. Replace the array parameters by pointers, and replace all indexing with pointer arithmetic.

```

// Copy n ints from src to dst using pointer parameters.
void copyInts(int dst[], const int src[] int n) {
    for (int i=0; i<n; i++)
    {
        // assign from current src element to current dst element
        dst[i] = src[i];
    }
}

```

Solution: index version

```

#include <iostream>
using namespace std;

// Copy n ints from src to dst using pointer parameters.
void copyInts(int dst[], const int src[], int n) {
    for (int i=0; i<n; i++)
    {
        // assign from current src element to current dst element
        dst[i] = src[i];
    }
}

int main()
{
    const int SIZE=3;
    int src[SIZE]={10,20,30};
    int dst[SIZE];
    copyInts(dst,src,SIZE);
    for(int i=0;i<SIZE;i++)
    {
        cout << src[i] << " = " << dst[i] << "\n";
    }
    return 0;
}

10 = 10
20 = 20
30 = 30

```

Pointer version:

```
#include <iostream>
using namespace std;

// Copy n ints from src to dst using pointer parameters.
void copyInts(int *dst, const int *src, int n) {
    for (int i=0; i<n; i++)
    {
        // assign from current src element to current dst element
        *(dst + i) = *(src + i);
    }
}

int main()
{
    const int SIZE=3;
    int src[SIZE]={10,20,30};
    int dst[SIZE];
    copyInts(dst,src,SIZE);
    for(int i=0;i<SIZE;i++)
    {
        cout << src[i] << " = " << dst[i] << "\n";
    }
    return 0;
}

10 = 10
20 = 20
30 = 30
```